

3.5 Terraform modules

i This next section is mainly a code-organisation refactor, not an infrastructure build. so the most important part is that we are not creating new AWS infrastructure. We are reorganising the Terraform code that already manages it.

Learning objective

What is a module?

A module is a reusable folder of terraform files instead of your root project containing everything, the root configuration calls smaller modules that each own one part of the system.

LUKE SAID: because you don't want to be constantly defining the same thing .e.g. most the lambdas policy rules were the same so they would stick that into a folder with a file called [lambdamain.tf](#) and lambdaoutput.tf or something then when we want to reference and get that police for a lambda we just put the name of th folder in the file we want it in and then you have the reusable code.

When is making one worth it, and when is it just extra indirection?

A Terraform module is worth creating when a group of resources belongs together, is reused, or makes the main configuration easier to understand. It becomes unnecessary indirection when the module is so small or simple that it makes the project harder to follow. For example, putting every single AWS resource into its own module would usually add complexity rather than improve the design.

How do input variables and outputs act as the module's interface?

Input variables are how the root Terraform configuration sends values into a module, while outputs are how the module sends useful values back out. This means the rest of the project does not need to know exactly how the module is built internally. For example, our SNS module receives the topic name as an input and returns the SNS topic ARN as an output.

What's the difference between a local module and one pulled from a registry?

A local module is stored inside your own project, for example `./modules/sns`, and is code that you control directly. A registry module is downloaded from an external source such as the Terraform Registry and is usually a reusable module created by another developer or organisation. Local modules are useful for organising your own infrastructure, while registry modules are useful when you want to reuse an existing solution instead of building everything yourself.

Task



- ~~Refactor your configuration into modules. Something like one for the API, one for the data layer, one for the messaging — pick boundaries you can justify~~
- ~~Same infrastructure, tidier code~~
- Terraform modules guide: [What Are Terraform Modules and How to Use Them - Tutorial](#)

al

okay you know what because this section isn't that important and I understand the way everything is built at the moment I dont want you changing the whole thing into modules. I just want you to pic 1 off the most insignificant parts in th project that could cause the least damage if broke and turn that into a module. so it sty shows that I can and have made a module.

Okay so I have choice out to make the SNS into a module, it is the safest choice and will cause the least amount of confusion. and it is important enough to be a module and to show my understanding of moduals.

So bellow is the new file format (none of the javascript changes the only changes are in the terraform files).

```
1 4-week-project/
2 |
3 |   terraform-week3/
4 |   |
5 |   |   lambda/
6 |   |   |
7 |   |   |   link-dynamodb/
8 |   |   |   |
9 |   |   |   |   controller/
10 |   |   |   |   |
11 |   |   |   |   |   readingsController.mjs
12 |   |   |   |   |
13 |   |   |   |   |   repositories/
14 |   |   |   |   |   |
15 |   |   |   |   |   |   readingsRepositories.mjs
16 |   |   |   |   |   |
17 |   |   |   |   |   |   services/
18 |   |   |   |   |   |   |
19 |   |   |   |   |   |   |   readingsService.mjs
20 |   |   |   |   |   |   |
21 |   |   |   |   |   |   |   index.mjs
22 |   |   |   |   |
23 |   |   |   |   |
24 |   |   |   |   |   excursion-handler/
25 |   |   |   |   |   |
26 |   |   |   |   |   |   controller/
27 |   |   |   |   |   |   |
28 |   |   |   |   |   |   |   excursionController.mjs
29 |   |   |   |   |   |   |
30 |   |   |   |   |   |   |   repositories/
31 |   |   |   |   |   |   |   |
32 |   |   |   |   |   |   |   |   excursionRepositories.mjs
33 |   |   |   |   |   |   |   |
34 |   |   |   |   |   |   |   |   services/
35 |   |   |   |   |   |   |   |   |
36 |   |   |   |   |   |   |   |   |   excursionServices.mjs
37 |   |   |   |   |   |   |   |   |
38 |   |   |   |   |   |   |   |   |   index.mjs
```

```

25 | terraform/
26 |
27 |   └─ modules/
28 |       └─ sns/
29 |           └─ main.tf
30 |           └─ variables.tf
31 |           └─ outputs.tf
32 |
33 |   └─ api-gateway.tf
34 |   └─ api-key.tf
35 |   └─ backend.tf
36 |   └─ dynamodb.tf
37 |   └─ iam.tf
38 |   └─ lambda.tf
39 |
40 | change
41 |   └─ main.tf
42 |   └─ output.tf
43 |
44 | reference change
45 |   └─ sns.tf
46 |   └─ sqs.tf
47 |
48 | change if required
49 |   └─ variables.tf
50 |
51 | variable
52 |   └─ .terraform.lock.hcl
53 |
54 |   └─ .gitignore
55 |   └─ README.md

```

← NEW
 ← NEW MODULE
 ← NEW
 ← NEW
 ← NEW
 ← unchanged
 ← unchanged
 ← unchanged
 ← unchanged
 ← mostly unchanged
 ← small SNS reference
 ← unchanged
 ← possibly small SNS
 ← CHANGED
 ← small SNS reference
 ← keeps existing SNS
 ← KEEP

The key thing is almost everything is unchanged and the only new folders are the following:

```

1 | terraform/
2 |   └─ modules/
3 |       └─ sns/
4 |           └─ main.tf
5 |           └─ variables.tf
6 |           └─ outputs.tf

```

[main.tf](#) = will hold the actual SNS topic resource:

[variable.tf](#) = these are just your normal

[output.tf](#) = This sends information **back out** of the module.

The current [SNS.tf](#) is not getting deleted we are simply re-purposing its function. Before the changes the SNS would create the SNS topics and connect them too SQS. Now it will be calling the module and will still connect the SNS topics.

The final important part to clarify before we get started is, terraform currently knows the SNS topic as “aws_sns_topic.excursions” and after awe move it into teh model folder its address will change too “module.sns.aws_sns_topic.excursions” Therefore we will also add a `moved` block to tell Terraform:

```

1 | moved {
2 |   from = aws_sns_topic.excursions
3 |   to   = module.sns.aws_sns_topic.excursions
4 | }

```

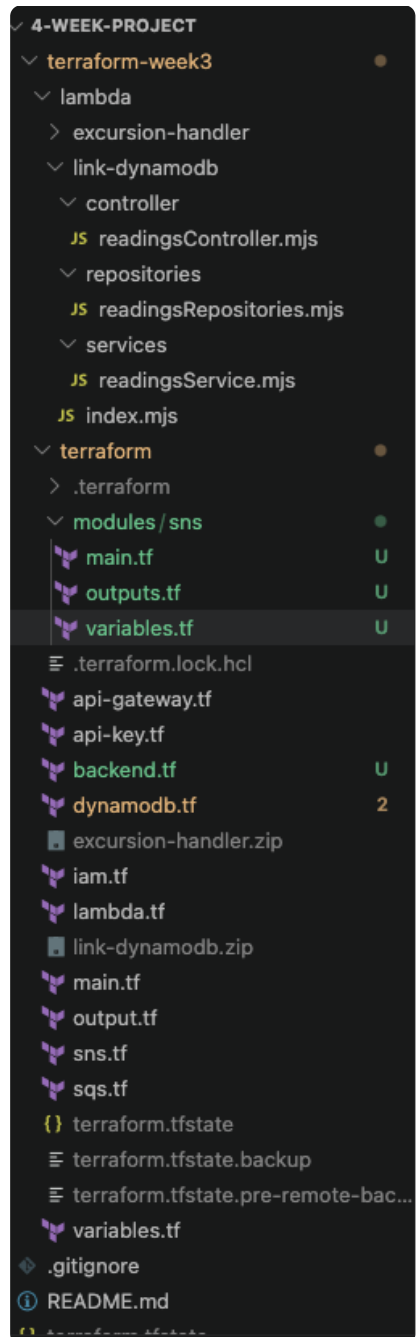
That means:

```

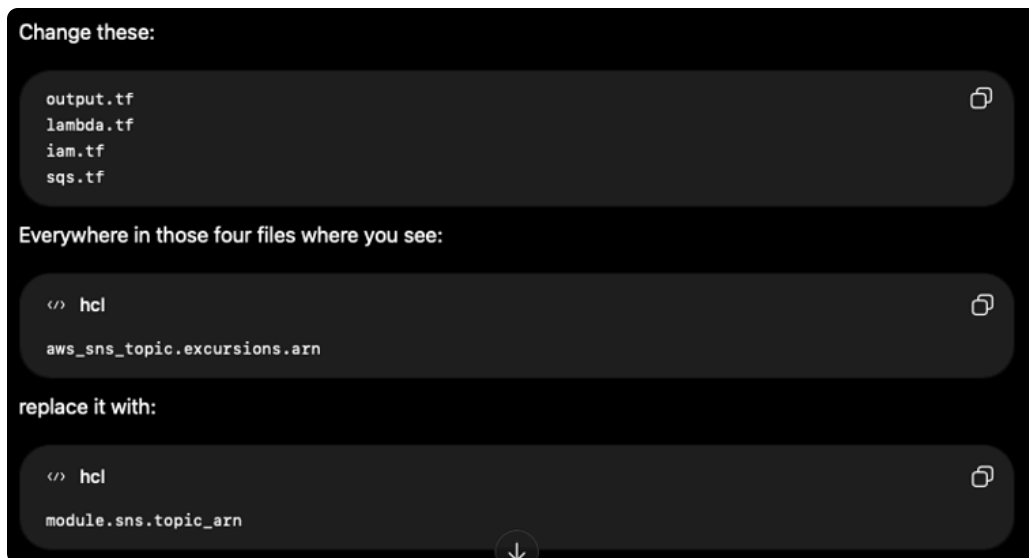
1 OLD Terraform address
2 aws_sns_topic.excursions
3   ↓
4   ↓ same AWS resource
5   ↓
6 NEW Terraform address
7 module.sns.aws_sns_topic.excursions

```

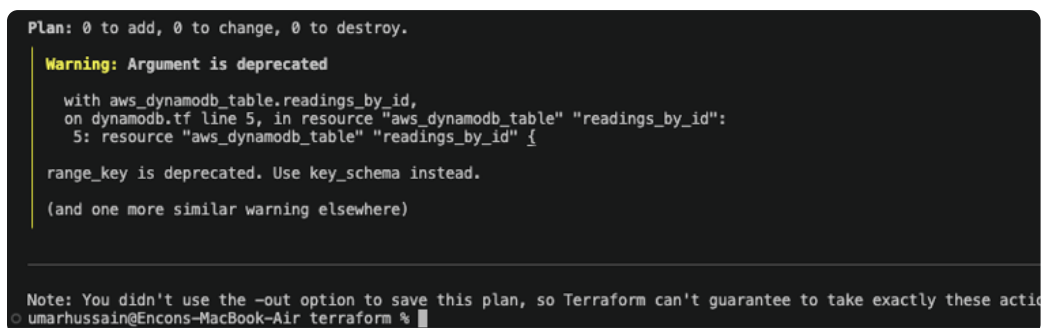
This is what should prevent Terraform from trying to destroy and recreate the topic.



The new file format



The minor changes that need to be made to the other files is show in the image above.



The image above shows the changes that I mean have worked. the new terraform structure has integrated perfectly with the old structure. this is shown in the image above as nothing has changed; nothing has been destroyed, created or changed. the module for the SNS has simply taken the place of the old [sns.tf](#) file. showing that I have successful created a module in terraform.

⚠ Questions / Queries / Pushback